

P1664R4

reconstructible_range - a concept for putting ranges back together

Published Proposal, 2021-06-15

Authors:

[JeanHeyd Meneide](#)

[Hannes Hauswedell](#)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Target:

C++23

Latest:

https://the-phd.dev/_vendor/future_cxx/papers/d1664.html

Abstract

This paper proposes new concepts to the Standard Library for ranges called Reconstructible Ranges for the purpose of ensuring a range/view broken down into its two iterators can be "glued" back together using an ADL-found function taking a tag, the range's iterator, and the range's sentinel.

Table of Contents

1	Revision History
1.1	Revision 4 - June 15th, 2021
1.2	Revision 3 - April 15th, 2021
1.3	Revision 2 - January 13th, 2020
1.4	Revision 1 - November 24th, 2019
1.5	Revision 0 - August, 5th, 2019
2	Motivation
2.1	Preservation of Properties
2.2	Compile-Time and Interoperability
3	Design
3.1	Range? Why not "Borrowed Range"?
3.2	Multiple, Safe Reconstruction Points
3.3	Opt-in?
3.4	Applicability
4	Deployment Experience
5	Impact
6	Proposed Changes
6.1	Feature Test Macro
6.2	Intent

- 6.3 Proposed Library Wording
- 6.3.1 Add a feature test macro `__cpp_lib_reconstructible_range`.
- 6.3.2 Insert into §24.2 Header `<ranges>` Synopsis [ranges.syn] a new customization point object in the inline namespace:
- 6.3.3 Insert into §24.4.2 Ranges [range.range]'s after paragraph 7, one additional paragraph:
- 6.3.4 Insert a new sub-clause "§24.3.13 `ranges::reconstruct` [`range.prim.recons`]", after "§24.3.12 `ranges::cdata` [`range.prim.cdata`]":
- 6.3.5 Add to "§21.4.3.1 `General`" [string.view.template.general] a `friend` function ADL extension point for `basic_string_view`:
- 6.3.6 Add a new subclause "§21.4.    `Range Reconstruction`" [string.view.reconstruct] in §21.4:
- 6.3.7 Add to "§22.7.3.1 `Overview`" [span.overview] a `friend` function ADL extension point for `span`:
- 6.3.8 Add a new subclause "§22.7.3.    `Range Reconstruction`" [span.reconstruct] in §22.7.3:
- 6.3.9 Add to "§24.5.4.1 `General`" [range.subrange.general] a `friend` function ADL extension point for `subrange`:
- 6.3.10 Add a new subclause "§24.5.4.    `Range Reconstruction`" [range.subrange.reconstruct] in §24.5.4:
- 6.3.11 Add to "§24.6.4.2 Class template `iota_view`" [range.iota] a `friend` function ADL extension point for `iota_view`:
- 6.3.12 Add one new paragraph to "§24.6.4.2 Class template `iota_view`" [range.iota]:
- 6.3.13 Add to "§24.6.2.2 Class template `empty_view`" [range.empty.view] a `friend` function ADL extension point for `empty_view`:
- 6.3.14 Add to "§24.7.6.2 Class template `transform_view`" [range.transform.view] a `friend` function ADL extension point for `transform_view`:
- 6.3.15 Add new paragraphs to "§24.6.4.2 Class template `transform_view`" [range.transform]:
- 6.3.16 Modify "§24.7.7.1 Overview " [range.take.overview] for `views::take`'s paragraph 2:
- 6.3.17 Modify "§24.7.9.1 Overview " [range.drop.overview] for `views::drop`'s paragraph 2:

7 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. Revision 4 - June 15th, 2021

- Rewrite most of the wording.
- Fix up the design section to always use an ADL Customization Point, all the time. The `is_class_v` and `is_enum_v` protects against most bad cases.

§ 1.2. Revision 3 - April 15th, 2021

- Vastly improve examples.
- Re-target motivation.
- Adjust fixes for things that have changed since C++20.

§ 1.3. Revision 2 - January 13th, 2020

- Improve wording and add more explanation in the design sections for the changes.

§ 1.4. Revision 1 - November 24th, 2019

- Change to `snake_case` concepts. (November 6th)
- Update wording to target [n4835]. (November 6th)
- Moved by LWG! 🐞 (November 6th)

C++20

```
template <typename T>
using span = quickcpplib::span<T>;

std::vector<int> vec{1, 2, 3, 4, 5};
span<int> s{vec.data(), 5};
span<int> v = s | views::drop(1) | views::take(10)
              | views::drop(1) | views::take(10);
```

✗ - Compilation error

```
template <typename T>
using span = quickcpplib::span<T>;

std::vector vec{1, 2, 3, 4, 5};
span<int> s{vec.data(), 5};
auto v = s | views::drop(1) | views::take(10)
          | views::drop(1) | views::take(10);
```

⚠ - Compiles, but `decltype(v)` is

```
ranges::take_view<ranges::drop_view<ranges::take_view<ranges::drop_view<span<int>,
dynamic_extent>>>>>
```

```
template <typename T>
using span = quickcpplib::span<T>;

std::vector<int> vec{1, 2, 3, 4, 5};
span<int> s{vec.data(), 5};
span<int> v = s | view::drop(1) | view::take(10)
              | view::drop(1) | view::take(10);
```

✓ - Compiles and works with 1

```
template <typename T>
using span = quickcpplib::span<T>;

std::vector vec{1, 2, 3, 4, 5};
span<int> s{vec.data(), 5};
auto v = s | views::drop(1) | views::take(10)
          | views::drop(1) | views::take(10);
```

✓ - Compiles and works with 1

```
quickcpplib::span<int>
```

```
std::u8string name = "APVEMI·M̂ET̂EIM גפג ·{טזל ·ז";
char16_t conversion_buffer[432];

std::u8string_view input(name);
std::span<char16_t> output(conversion_buffer, 432);

auto encoding_result = ztd::text::transcode(input, output);
// compiler error here !!
std::u8string_view unprocessed_code_units
    = encoding_result.input;
std::span<char16_t> unconsumed_output
    = encoding_result.output;
```

✗ - Compilation error

```
std::u8string name = "APVEMI·M̂ET̂EIM גפג ·{טזל ·ז";
char16_t conversion_buffer[432];

std::u8string_view input(name);
std::span<char16_t> output(conversion_buffer, 432);

auto encoding_result = ztd::text::transcode(input, output);
auto unprocessed_code_units = encoding_result.input;
auto unconsumed_output = encoding_result.output;
```

```
std::u8string name = "
char16_t conversion_bu

std::u8string_view inp
std::span<char16_t> ou

auto encoding_result =
std::u8string_view unp
    = encoding_res
std::span<char16_t> un
    = encoding_res
```

✓ - Compiles and works, types

```
std::u8string name = "
char16_t conversion_bu

std::u8string_view nam
std::span<char16_t> ou

auto encoding_result =
auto unprocessed_code_
auto unconsumed_output
```

⚠ - Compiles, but <code>decltype(unprocessed_code_units)</code> is <code>ranges::subrange<std::u8string_view::iterator, std::u8string_view::iterator></code> and <code>decltype(unconsumed_output)</code> is <code>ranges::subrange<std::span<char16_t, std::dynamic_extent>::iterator, std::span<char16_t, std::dynamic_extent>::iterator></code>	✓ - Compiles and works, where <code>std::u8string_view</code> and <code>std::span<char16_t></code>.
---	--

Currently in C++, there is no Generic ("with a capital G") way to take a range apart with its iterators and put it back together. That is, the following code is not guaranteed to work:

```
template <typename Range>
auto operate_on_and_return_updated_range (Range&& range) {
    using uRange = std::remove_cvref_t<Range>;
    if (std::ranges::empty(range)) {
        // ⚠!
        // ... the below errors
        return uRange(std::forward<Range>(range));
    }
    /* perform some work with the
    iterators or similar */
    auto first = std::ranges::begin(range);
    auto last = std::ranges::end(range);
    if (*first == u'\0xEF') {
        // ...
        std::advance(first, 3);
        // ...
    }
    // ... algorithm finished,
    // return the "updated" range!

    // ⚠!
    // ... but the below errors
    return uRange(std::move(first), std::move(last));
}

int main () {
    std::string_view meow_view = "나는 유리를 먹을 수 있어요. 그래도 아프지 않아요";
    // this line will error in C++17, or in compatibility
    // libraries
    std::string_view sub_view
        = operate_on_and_return_updated_range(meow_view);
    return 0;
}
```

The current fix is to employ `std::ranges::subrange<I, S, K>` to return a generic subrange:

```
template <typename Range>
auto operate_on_and_return_updated_range (Range&& range) {
    using uRange = std::remove_cvref_t<Range>;
    using I = std::iterator<uRange>;
    using S = std::Sentinel<uRange>;
    using Result = std::ranges::subrange<I, S>;
    if (std::ranges::empty(range)) {
        return Result(std::forward<Range>(range));
    }
    // perform some work with the
    // iterators or similar
    auto first = std::ranges::begin(range);
    auto last = std::ranges::end(range);
    if (*first == u'\0xEF') {
        // ...
        std::advance(first, 3);
        // ...
    }
}
```

```

    // ... algorithm finished,
    // return the "updated" range!

    // now it works!
    return Result(std::move(first), std::move(last));
}

int main () {
    std::string_view meow_view = "나는 유리를 먹을 수 있어요. 그래도 아프지 않아요";
    auto sub_view
        = operate_on_and_return_updated_range(meow_view);
    // decltype(sub_view) ==
    //   std::ranges::subrange<std::string_view::iterator, std::string_view::iterator>
    //   which is nowhere close to ideal.
    return 0;
}

```

This makes it work with any two pair of iterators, but quickly becomes undesirable from an interface point of view. If a user passes in a `quickcpplib::span<T, Extent>` or a `llvm::StringRef` that interface and information is entirely lost to the user of the above function. `std::ranges::subrange<Iterator, Sentinel, Kind>` does not -- and cannot/should not -- mimic the interface of the view it was created from other than what information comes from its iterators: it is the barebones idea of a pair-of-iterators/iterator-sentinel style of range. This is useful in the generic sense that if a library developer must work with iterators, they can always rely on creation of a `std::ranges::subrange` of the iterator and sentinel.

§ 2.1. Preservation of Properties

Unfortunately, always using `std::ranges::subrange` decreases usability for end users. Users who have, for example a `llvm::StringRef`, would prefer to have the same type after calling an algorithm. These types have functions and code that are purpose-made for `string_view`-like code: losing that intent means needing to reconstruct that from the ground up all over again. There is little reason why the original type needs to be discarded if it supports being put back together from its iterators.

The break-it-apart-and-then-generic-`subrange` approach also discards any range-specific storage optimizations and layout considerations, leaving us with the most bland kind of range similar to the "pair of iterators" model. Consider, for example, `llfio::span<T>`. The author of the Low Level File IO (LLFIO) library, Niall Douglass, has spent countless days submitting pull requests and change requests to MSVC, GCC, and Clang's standard libraries to ask them to change their structural layout to match things like the `iovec` of their platform or their asynchronous buffer span types. This optimization matters because it realizes the performance difference between having to individually transformed a container of `spans` to a more suitable type, or being able to simply `memcpy` the desired sequences of spans into the underlying asynchronous operations. If range operations were to be performed on the `llfio::span<T>` type, a `std::ranges::subrange` would hold, in most cases, two iterators rather than an iterator and a size. This completely eliminates the `memcpy` optimization. This is not the only place this happens, however: other types have different storages requirements that are not appropriately captured by their iterators in the most general sense, but may benefit from reconstruction and desired type information for the target range they should be constructed into.

§ 2.2. Compile-Time and Interoperability

Compilation time goes up as well in the current paradigm. Users must spawn a fresh `std::ranges::subrange<I, S, K>` for every different set of iterator/sentinel/kind triplet, or handle deeply nested templates in templates as the input types. This makes it impossible to compile interfaces as dynamic libraries without having to explicitly materialize or manually cajole a `std::ranges::subrange` into something more palatable for the regular world.

There is also a problem where there are a wide variety of ranges that could conceivably meet this criterion, but do not. The author of this paper was not the only one to see utility for this. [\[p1739r4\]](#) does much the same that this paper does, without the introduction of a concept to formalize the behavior it presents. In particular, it selects views which can realistically have their return types changed to match the input range and operations being performed (or a similarly powerful alternative) by

asking whether they can be called with a function called `reconstruct` from a subrange of the iterators with the expressions acted upon.

- Ranges should be reconstructible from their iterators (or subrange of their iterators) where applicable;
- and, reconstructible ranges serve a useful purpose in generic algorithms, including not losing information and returning it in a much more cromulent and desirable form.

Therefore, this paper formalizes that concept and provides an **opt-in, user-overridable way** to return a related "reconstructed" type from an tag, an iterator, and a sentinel.

§ 3. Design

The design is given in 3 concepts added to the standard:

```
template <class It, class Sen = It>
concept iterator_reconstructible_range =
(std::is_class_v<It> || std::is_class_v<Sen> || std::is_enum_v<It> || std::is_enum_v<Sen>)
&& requires (It first, Sen last) {
    reconstruct(
        std::forward<It>(first),
        std::forward<Sen>(last)
    );
};

template <class R,
    class It = ranges::iterator_t<remove_reference_t<R>>,
    class Sen = ranges::sentinel_t<remove_reference_t<R>>>
concept reconstructible_range =
std::ranges::range<R>
&& requires (It first, Sen last) {
    reconstruct(
        in_place_type<remove_cvref_t<R>>,
        std::move(first),
        std::move(last)
    );
};

template <class R, class Tag = remove_cvref_t<R>,
    class It = ranges::iterator_t<remove_reference_t<R>>,
    class Sen = ranges::sentinel_t<remove_reference_t<R>>>
concept range_reconstructible_range =
std::ranges::range<R>
&& requires (R range, It first, Sen last) {
    reconstruct(
        in_place_type<Tag>,
        std::forward<R>(range),
        std::forward<It>(first),
        std::forward<Sen>(last)
    );
};
```

It is the formalization that a range can be reconstructed from its necessary components. The concepts are a means of implementing the desired Customization Point Object (CPO) which will be called `std::ranges::reconstruct`. The 3 forms allow for 3 levels of construction, here each one defers to the next version by removing one layer of "information" until finally following back to the "default" form:

- the desired type to reconstruct ("tag"), the range, the iterator, and the sentinel;
- the desired type to reconstruct ("tag"), the iterator, and the sentinel, after stripping out the range if the previous `reconstruct` did not find anything;

- the iterator and the sentinel, after dropping the tag if the previous `reconstruct` attempt did not work and making sure at least one of `std::is_class_v<iterator>`, `std::is_class_v<sentinel>`, `std::is_enum_v<iterator>`, or `std::is_enum_v<sentinel>` is true; and finally,
- `std::ranges::subrange<decltype(iterator), decltype(sentinel)>` if nothing else matches.

This allows a developer to put a range back together at various levels of fidelity from its parts and pieces, giving us the ability to propagate the input type's properties after modifying its iterators for some underlying work, algorithm or other effect. This concept is also the basis of the idea behind [\[p1739r4\]](#), which was accepted in a lesser form for C++20 with the intent of this paper following up on it.

§ 3.1. Range? Why not "Borrowed Range"?

Previously, we required that the ranges being reconstructed modeled `borrowed_range`, since that was an accurate indicator that the iterators and sentinel could potentially outlive the range itself. This was to prevent some issues in a previous design that used constructors. Since that is no longer based on constructors and with no chance of false positives, we no longer need to include that limitation. `borrowed_range` is also too restrictive, as there are some cases where the resulting range is non-borrowed, but could be successfully reconstructed from some of its pieces. The concept is opt-in now by way of declaring an ADL-found `reconstruct` function, any type can effectively decide for itself whether it could reconstruct properly. Note that this opens up usages for things that, if we had used `borrowed_range`, would not have been reconstructible. Consider the following expression:

```
std::vector<int> vec{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
std::span<int> some_span(vec);
some_span | views::transform(f) | views::take(5)
```

This particular case is reconstructible as `span<T>` is reconstructible (and `span<T>` is reconstructible because it is already a properly borrowed type). The original transformation view can be reconstructed here without the need to keep the `take_view` intermediate wrapped around the two previous transformations, as the transform can be ignored on the first five elements by advancing the given `some_span`'s `begin()` by 5 elements. A smart `reconstruct` call for `take_view` could conceivably do this.

§ 3.2. Multiple, Safe Reconstruction Points

Consider a hypothetical `c_string_view` type. It is impossible to, from normal `const char*` iterator pair ascertain the range is null-terminated and is thus a "C string". Therefore, a reconstruct for `c_string_view` would not return a `c_string_view`, but a normal `string_view`.

On the other hand, if there was a `c_string_sentinel` sentinel with a `const char*` iterator that was preserved through a series of actions and algorithms, that kind of information would allow us to return a `c_string_view` from a reconstruction operation.

The `reconstruct` extension point supports both of these scenarios:

```
class c_string_view {
    /* blah... */

    // reconstruct: with [const char*, const char*) iterators
    // no guarantee it's null terminated: decay to string_view
    friend constexpr std::string_view reconstruct (
        std::in_place_type_t<c_string_view>,
        const char* first, const char* last) noexcept {
        return std::string_view(first, last);
    }

    // reconstruct: with static [const char*, c_string_sentinel)
    // static guarantee by type: return a c_string_view
    friend constexpr c_string_view reconstruct (
        std::in_place_type_t<c_string_view>,
        const char* first, c_string_sentinel) noexcept {
```

```

        return c_string_view(first);
    }
};

```

This is a level of flexibility that is better than the Revision 0 constructor-based design, and can aid in providing better static guarantees while decaying gracefully in other situations.

§ 3.3. Opt-in?

Not all ranges can meet this requirement. Some ranges contain state which cannot be trivially propagated into the iterators, or state that cannot be reconstructed from the iterator/sentinel pair itself. However, most of the common ranges representing unbounded views, empty views, iterations viewing some section of non-owned storage, or similar can all be reconstructed from their iterator/iterator or iterator/sentinel pair.

For example `std::ranges::single_view` contains a [exposition *semiregular-box* template type](#) (`ranges::semi.wrap`) which holds a value to iterate over. It would not be possible to reconstruct the exact same range (e.g., iterators pointing to the exact same object) with the semi-regular wrapper. Ranges should have the ability to select this syntax without having it happen to their type implicitly, or as for more information.

This is why there are 3 levels to opt-in to support. If a user defines the 4-argument form which takes a `(tag, range, iterator, sentinel)` quadruplet, then they can use as much information may be available in the range, iterator, and sentinel to reproduce the original range. If they do not need that much information, they can opt-in to the 3-argument version which only takes the `(tag, iterator, sentinel)` triplet. And, finally, the most brief and sophisticated form is the `(iterator, sentinel)` form, which can reconstruct a range using information from only the iterator, only the sentinel, or both.

If the range opts-in to nothing, then there will always be a default return: `std::ranges::subrange<Iterator, Sentinel>`. This is effectively what any algorithm today already returns (sometimes it uses a hand-made pair type, likely because some iterator-pair usages may require the size to be stored and neither the iterator nor the sentinel can provide the size).

§ 3.4. Applicability

There are many ranges to which this is applicable, but only a handful in the standard library need or satisfy this. If [\[p1391r2\]](#) and [\[p1394r2\]](#) are accepted (they were), then the two most important view types -- `std::span<T, Extent>` and `std::basic_string_view<Char, Traits>` -- will model the `_spirit_` of the concept (but lack the customization point to make it generic). `std::ranges::subrange<Iterator, Sentinel, Kind>` already fits this as well. By formalizing concepts in the standard, we can dependably and reliably assert that these properties continue to hold for these ranges. We intend to add a `constexpr friend reconstruct` function to all of the non-`ranges::subrange` types.

There are also upcoming ranges from [\[range-v3\]](#) and elsewhere that could model this concept:

- [\[p1255r4\]](#)'s `std::ranges::ref_maybe_view`;
- [\[p0009r9\]](#)'s `std::mdspan`;
- and, soon to be proposed by this author for the purposes of output range algorithms, [\[range-v3\]](#)'s `ranges::unbounded_view`.

And there are further range adaptor closure objects that could make use of this concept:

- `views::slice`, `views::take_exactly`, `views::drop_exactly` and `views::take_last` from [\[range-v3\]](#)

Note that these changes will greatly aid other algorithm writers who want to preserve the same input ranges. In the future, the standard may provide an `ranges::reconstruct(...)` algorithm for these types.

§ 4. Deployment Experience

This change was motivated by Hannes Hauswedell's [\[p1739r4\]](#) and became very important for the ranges work done with text encoding. There is a C++17 implementation [at the soasis/text repository here](#), which is mimicking the interface needed

for [\[p1629r1\]](#). It is meant to solve the deployment issues with P1739's merging into the Standard.

§ 5. Impact

As a feature that is opt-in thanks to the `ranges::reconstruct` Customization Point Object design, no currently created range or previously made range is affected.

Furthermore, this is a new and separate set of concepts. It is not to be added to the base `Range` concept, or added to any other concept. It is to be applied separately to the types which can reasonably support it for the benefit of algorithms and code which can enhance the quality of their implementation.

Finally, Hannes Hauswedell's [\[p1739r4\]](#) with the explicit intention to mark certain ranges as reconstructible by hardcoding their behavior into the standard and come back with an opt-in fix during the C++23 cycle. This paper completes that promise.s

§ 6. Proposed Changes

The following wording is relative to the latest C++ Draft paper.

§ 6.1. Feature Test Macro

This paper results in a concept to help guide the further development of standard ranges and simplify their usages in generic contexts. There is one proposed feature test macro, `__cpp_lib_reconstructible_range`, which is to be input into the standard and then explicitly updated every time a `reconstruct` from a is added to reflect the new wording. We hope that by putting this in the standard early, most incoming ranges will be checked for compatibility with `pair_reconstructible_range` and `reconstructible_range`.

§ 6.2. Intent

The intent of this wording is to provide greater generic coding guarantees and optimizations by allowing for a class of ranges and views that model the new exposition-only definitions of a reconstructible range:

- add a new feature test macro for reconstructible ranges to cover constructor changes;
- add a new customization point object for `ranges::reconstruct`;
- add the customization point to many different ranges that can be reconstructed;
- and, add two new concepts to [\[range.req\]](#).

§ 6.3. Proposed Library Wording

§ 6.3.1. Add a feature test macro `__cpp_lib_reconstructible_range`.

§ 6.3.2. Insert into §24.2 Header `<ranges>` Synopsis [[ranges.syn](#)] a new customization point object in the inline namespace:

```
namespace std::ranges {  
    inline namespace unspecified {  
        ...  
  
        nline constexpr nspecified reconstruct = unspecified;  
  
        ...  
    }  
}
```

§ 6.3.3. Insert into §24.4.2 Ranges [\[range.range\]](#)'s after paragraph 7, one additional paragraph:

⁸The concepts `iterator_reconstructible_range`, `reconstructible_range` and `range_reconstructible_range` concepts describe the requirements on ranges that are efficiently constructible from certain iterator and sentinel types, alongside possible additional information from the range itself.

```
template <class It, class Sen = It>
    concept iterator_reconstructible_range =
        (std::is_class_v<It> || std::is_class_v<Sen>
         || std::is_enum_v<It> || std::is_enum_v<Sen>)
    && requires (It first, Sen last) {
        reconstruct(
            std::forward<It>(first),
            std::forward<Sen>(last)
        );
    };

template <class R,
    class It = ranges::iterator_t<remove_reference_t<R>>,
    class Sen = ranges::sentinel_t<remove_reference_t<R>>>
    concept reconstructible_range =
        std::ranges::range<R>
    && requires (It first, Sen last) {
        reconstruct(
            in_place_type<remove_cvref_t<R>>,
            std::move(first),
            std::move(last)
        );
    };

template <class R, class Tag = remove_cvref_t<R>,
    class It = ranges::iterator_t<remove_reference_t<R>>,
    class Sen = ranges::sentinel_t<remove_reference_t<R>>>
    concept range_reconstructible_range =
        std::ranges::range<R>
    && requires (R range, It first, Sen last) {
        reconstruct(
            in_place_type<Tag>,
            std::forward<R>(range),
            std::forward<It>(first),
            std::forward<Sen>(last)
        );
    };
};
```

⁹ Let `r` be a range with type `R` and `T` be some type.

9.1 — Let `it Sen re_range` be the result of `ranges::reconstruct(ranges::begin(r), ranges::end(r))` if such an expression is well-formed. `r` models `ranges::iterator_reconstructible_range` if

- `ranges::begin(r) == ranges::begin(it Sen re_range)` is true, and
- `ranges::end(r) == ranges::end(it Sen re_range)` is true.

9.2 — Let `re_range` be the result of `ranges::reconstruct(in_place_type<remove_cvref_t<T>>, ranges::begin(r), ranges::end(r))` if such an expression is well-formed. `r` models `ranges::reconstructible_range` if

- `ranges::begin(r) == ranges::begin(re_range)` is true, and
- `ranges::end(r) == ranges::end(re_range)` is true.

9.3 — Let `range re_range` be the result of `ranges::reconstruct(in_place_type<remove_cvref_t<T>>, r, ranges::begin(r), ranges::end(r))` if such an expression is well-formed. `r` models `range_reconstructible_range` if

ranges::end(r)), if such an expression is well-formed. Then range_re_range models ranges::range_reconstructible_range if

— ranges::begin(r) == ranges::begin(range_re_range) is true, and
— ranges::end(r) == ranges::end(range_re_range) is true.

- § 6.3.4. Insert a new sub-clause "§24.3.13 **ranges::reconstruct** [range.prim.recons]", after "§24.3.12 **ranges::cdata** [range.prim.cdata]":

24.3.12 **ranges::reconstruct**

[range.prim.recons]

¹ The name **reconstruct** denotes a customization point object.

² The expression **ranges::reconstruct**(*I*, *S*) for some sub-expressions *I* and *S* is expression-equivalent to:

(2.1) reconstruct(std::forward<decltype(*I*)>(*I*), std::forward<decltype(*S*)>(*S*)) if either remove_cvref_t<decltype(*I*)> or remove_cvref_t<decltype(*S*)> are class or enumeration types, it is a valid expression, and decltype(*I*), and decltype(*S*) model iterator_reconstructible_range.

(2.2) Otherwise, ranges::subrange<remove_cvref_t<decltype(*I*)>, remove_cvref_t<decltype(*S*)>> (std::forward<decltype(*I*)>(*I*), std::forward<decltype(*S*)>(*S*)) if it is a valid expression.

(2.3) Otherwise, **ranges::reconstruct**(*I*, *S*) is ill-formed.

³ The expression **ranges::reconstruct**(in_place_type<*R*>, *I*, *S*) for some type *R* and some sub-expressions *I* and *S* is expression-equivalent to:

(2.1) reconstruct(in_place_type<*R*>, std::forward<decltype(*I*)>(*I*),
std::forward<decltype(*S*)>(*S*)) if it is a valid expression and *R*, decltype(*I*), and decltype(*S*) model range_reconstructible_range.

(2.2) Otherwise, ranges::reconstruct(std::forward<decltype(*I*)>(*I*),
std::forward<decltype(*S*)>(*S*)).

⁴ The expression **ranges::reconstruct**(in_place_type<*R*>, *SR*, *I*, *S*) for some type *R*, and some sub-expressions *SR*, *I*, and *S* is expression-equivalent to:

(2.1) reconstruct(in_place_type<*R*>, std::forward<decltype(*SR*)>(*SR*),
std::forward<decltype(*I*)>(*I*), std::forward<decltype(*S*)>(*S*)) if it is a valid expression and decltype(*SR*), *R*, decltype(*I*), and decltype(*S*) model range_reconstructible_range.

(2.2) Otherwise, ranges::reconstruct(in_place_type<*R*>, std::forward<decltype(*I*)>(*I*),
std::forward<decltype(*S*)>(*S*)).

- § 6.3.5. Add to "§21.4.3.1 **General**" [string.view.template.general] a **friend** function ADL extension point for **basic_string_view**:

```
// [string.view.reconstructl  
friend constexpr basic_string_view  
reconstruct(const_iterator first, const_iterator last) noexcept;
```

- § 6.3.6. Add a new subclause "§21.4.◆◆◆◆ Range Reconstruction" [string.view.reconstruct] in §21.4:

21.4.◆◆◆◆ Range Reconstruction

[string.view.reconstruct]

```
friend constexpr basic_string_view  
reconstruct(in_place_type_t<basic_string_view>, const_iterator first, const_itera
```

¹ Returns: `basic_string(std::move(first), std::move(last))`.

- § 6.3.7. Add to "§22.7.3.1 Overview" [span.overview] a **friend** function ADL extension point for `span`:

```
// [span.reconstruct].  
template <class It, class End>  
friend constexpr auto  
reconstruct(in_place_type_t<span>, It first, End last) noexcept;
```

- § 6.3.8. Add a new subclause "§22.7.3.◆◆◆◆ Range Reconstruction" [span.reconstruct] in §22.7.3:

22.7.3.◆◆◆◆ Range Reconstruction

[span.reconstruct]

```
template <class It, class End>  
friend constexpr auto  
reconstruct(in_place_type_t<span>, It first, End last) noexcept;
```

¹ Constraints: Let `U` be `remove_reference_t<iter_reference_t<It>>`.

(1.1) — `is_convertible_v<U(*)[], element_type(*)[]>` is true. [Note: The intent is to allow only qualification conversions of the iterator reference type to element type. — end note]

(1.2) — `It` satisfies `contiguous_iterator`.

(1.3) — `End` satisfies `sized_sentinel_for<It>`.

(1.4) — `is_convertible_v<End, size_t>` is false.

² Let `MaybeExtent` be an implementation-defined constant expression of type `size_t`. If it is not equal to `dynamic_extent`, then `MaybeExtent` shall be a value equal to `distance(first, last)`.

³ Returns: `span<ElementType, MaybeExtent>(std::move(first), std::move(last))`.

⁴ Remarks: the return type may be promoted to a `span` with a static extent if the implementation is capable of deriving such information from the given iterator and sentinel.

- § 6.3.9. Add to "§24.5.4.1 General" [range.subrange.general] a **friend** function ADL extension point for `subrange`:

```
friend constexpr auto  
reconstruct(in_place_type_t<subrange>, I first, E last) noexcept;
```

- § 6.3.10. Add a new subclause "§24.5.4.     **Range Reconstruction** " [range.subrange.reconstruct] in §24.5.4:

22.7.3. **Range Reconstruction**

[range.subrange.reconstruct]

```
friend constexpr subrange  
reconstruct(in_place_type_t<subrange>, I first, S last) noexcept(see-below);
```

¹ Returns: `subrange(std::move(first), std::move(last))`.

² Throws: *Anything from evaluating the Returns. Otherwise, nothing.*

- § 6.3.11. Add to "§24.6.4.2 Class template `iota_view`" [range.iota] a **friend** function ADL extension point for `iota_view`:

```
friend constexpr iota_view  
reconstruct(iterator first, sentinel last) noexcept;
```

- § 6.3.12. Add one new paragraph to "§24.6.4.2 Class template `iota_view`" [range.iota]:

```
friend constexpr iota_view  
reconstruct(iterator first, sentinel last) noexcept;
```

  Returns: `iota_view(std::move(first), std::move(last))`.

- § 6.3.13. Add to "§24.6.2.2 Class template `empty_view`" [range.empty.view] a **friend** function ADL extension point for `empty_view`:

```
friend constexpr empty_view  
reconstruct(iterator, sentinel) noexcept {  
    return empty_view();  
}
```

- § 6.3.14. Add to "§24.7.6.2 Class template `transform_view`" [range.transform.view] a **friend** function ADL extension point for `transform_view`:

```
friend constexpr transform_view  
reconstruct(iterator, sentinel) noexcept(see-below);
```

§ 6.3.15. Add new paragraphs to "§24.6.4.2 Class template `transform_view`" [[range.transform](#)]:

```
friend constexpr transform_view  
reconstruct(iterator first, sentinel last) noexcept(see-below);
```

◆◆ Constraints: `V` models `reconstructible_range`.

◆◆ Returns: `transform_view(ranges::reconstruct(std::move(first).current, std::move(last).end), *first.parent->fun_)`.

◆◆◆ Throws: Anything from evaluating the *Returns*. Otherwise, nothing.

```
friend constexpr transform_view  
reconstruct(in_place_type_t<transform_view>, transform_view original,  
iterator first, sentinel last) noexcept(see-below);
```

◆◆ Constraints: `V` models `reconstructible_range`.

◆◆ Returns: `transform_view(ranges::reconstruct(std::move(first).current, std::move(last).end), std::move(*original.fun_))`.

◆◆◆ Throws: Anything from evaluating the *Returns*. Otherwise, nothing.

§ 6.3.16. Modify "§24.7.7.1 Overview" [[range.take.overview](#)] for `views::take`'s paragraph 2:

² The name `views::take` denotes a range adaptor object (`[range.adaptor.object]`). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::take(E, F)` is ill-formed. Otherwise, the expression `views::take(E, F)` is expression-equivalent to:

~~(2.1) — If `T` is a specialization of `ranges::empty_view` (`[range.empty.view]`), then `((void) F, decay-copy(E))`.~~

~~(2.2) — Otherwise, if `T` models `random_access_range` and `sized_range` and is~~

~~(2.2.1) — a specialization of `span` (`[views.span]`) where `T::extent == dynamic_extent`;~~

~~(2.2.2) — a specialization of `basic_string_view` (`[string.view]`);~~

~~(2.2.3) — a specialization of `ranges::iota_view` (`[range.iota.view]`), or~~

~~(2.2.4) — a specialization of `ranges::subrange` (`[range.subrange]`);~~

~~then `T{ranges::begin(E), ranges::begin(E) + min<D>(ranges::size(E), F)}`, except that `E` is evaluated only once.~~

~~(2.3) — Otherwise, `ranges::take_view{E, F}`.~~

(2.1) — If `T` is a specialization of `ranges::empty_view` (`[range.empty.view]`), then `((void) F, decay-copy(E))`.

(2.2) — Otherwise, if `T` models both `random_access_range` and `sized_range` and `T`:

(2.2.1) — models `range_reconstructible_range`, or

(2.2.2) — models `reconstructible_range`, or

(2.2.3) — has an iterator `t<T>` and a sentinel `t<T>` that models `iterator_reconstructible_range`.

then `ranges::reconstruct(in_place_type<T>, E, ranges::begin(E), ranges::begin(E) + min<D>(ranges::size(E), F))`, except that `E` is evaluated only once.

(2.3) — Otherwise, `ranges::take_view{E, F}`.

§ 6.3.17. Modify "§24.7.9.1 Overview" [range.drop.overview] for `views::drop`'s paragraph 2:

² The name `views::drop` denotes a range adaptor object (`[range.adaptor.object]`). Let `E` and `F` be expressions, let `T` be `remove_cvref_t<decltype((E))>`, and let `D` be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::drop(E, F)` is ill-formed. Otherwise, the expression `views::drop(E, F)` is expression-equivalent to:

~~(2.1) — If `T` is a specialization of `ranges::empty_view` (`[range.empty.view]`), then `((void) F, decay-copy(E))`.~~

~~(2.2) — Otherwise, if `T` models `random_access_range` and `sized_range` and is~~

~~(2.2.1) — a specialization of `span` (`[views.span]`) where `T::extent == dynamic_extent`;~~

~~(2.2.2) — a specialization of `basic_string_view` (`[string.view]`);~~

~~(2.2.3) — a specialization of `ranges::iota_view` (`[range.iota.view]`), or~~

~~(2.2.4) — a specialization of `ranges::subrange` (`[range.subrange]`);~~

~~then `T{ranges::begin(E) + min<D>(ranges::size(E), F), ranges::end(E)}`, except that `E` is evaluated only once.~~

~~(2.3) — Otherwise, `ranges::drop_view{E, F}`.~~

(2.1) — If `T` is a specialization of `ranges::empty_view` (`[range.empty.view]`), then `((void) F, decay-copy(E))`.

(2.2) — Otherwise, if `T` models both `random_access_range` and `sized_range`, and `T`:

(2.2.1) — models `range_reconstructible_range`, or

(2.2.2) — models `reconstructible_range`, or

(2.2.3) — has an iterator `t<T>` and a sentinel `t<T>` that models `iterator_reconstructible_range`.

then `ranges::reconstruct(in_place_type<T>, E, ranges::begin(E) + min<D>(ranges::size(E), F), ranges::end(E))`, except that `E` is evaluated only once.

(2.3) — Otherwise, `ranges::drop_view{E, F}`.

§ 7. Acknowledgements

Thanks to Corentin Jabot, Christopher DiBella, and Hannes Hauswedell for pointing me to [p1035r7] and [p1739r4] to review both papers and combine some of their ideas in here. Thanks to Eric Niebler for prompting me to think of the generic, scalable solution to this problem rather than working on one-off fixes for individuals views.

Thank you to Oktal, Anointed of ADL, Blessed Among Us, and Morwenn, the ever-watching Code Guardian for suggesting improvements to the current concept form.

<https://cplusplus.github.io/LWG/issue3407>

§ References

§ Informative References

[N4835]

Richard Smith. [Working Draft, Standard for Programming Language C++](#). 8 October 2019. URL: <https://wg21.link/n4835>

[P0009R9]

H. Carter Edwards, Bryce Adelstein Lelbach, Daniel Sunderland, D. S. Hollman, Christian Trott, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Mark Hoemmen. [mdspan: A Non-Owning Multidimensional Array](#)

[Reference](https://wg21.link/p0009r9), 20 January 2019. URL: <https://wg21.link/p0009r9>

[P1035R7]

Christopher Di Bella, Casey Carter, Corentin Jabot. [Input Range Adaptors](#), 2 August 2019. URL: <https://wg21.link/p1035r7>

[P1255R4]

Steve Downey. [A view of 0 or 1 elements: view::maybe](#), 17 June 2019. URL: <https://wg21.link/p1255r4>

[P1391R2]

Corentin Jabot. [Range constructor for std::string_view](#), 17 June 2019. URL: <https://wg21.link/p1391r2>

[P1394R2]

Corentin Jabot, Casey Carter. [Range constructor for std::span](#), 17 June 2019. URL: <https://wg21.link/p1394r2>

[P1629R1]

JeanHeyd Meneide. [Transcoding the world - Standard Text Encoding](#), 2 March 2020. URL: <https://wg21.link/p1629r1>

[P1739R4]

Hannes Hauswedell. [Avoid template bloat for safe_ranges in combination with 'subrange-y' view adaptors.](#), 1 March 2020. URL: <https://wg21.link/p1739r4>

[RANGE-V3]

Eric Niebler; Casey Carter. [range-v3](#), June 11th, 2019. URL: <https://github.com/ericniebler/range-v3>